PHP Developer Core Reference

1. PHP Fundamentals

PHP (Hypertext Preprocessor) is a server-side scripting language designed primarily for web development. Scripts are executed on the server, and the result is returned to the browser as plain HTML. PHP files have a `.php` extension and can contain text, HTML, CSS, JavaScript, and PHP code. A PHP developer must master variables, control structures, strings, arrays, and loops to build dynamic applications.

```
<?php
// Variables start with $, are case-sensitive, and loosely typed
$greeting = "Hello, World!";
$version = 8.2;
$isActive = true;

// Basic conditional
if ($isActive) {
    echo $greeting . " PHP Version: " . $version;
}
?>
```

2. Strings in PHP

What is a String? A string is a sequence of characters, like "Hello". In PHP, strings can be enclosed in single quotes (`' '`) or double quotes (`" "`). Double quotes allow for variable parsing (interpolation) directly inside the string, while single quotes treat everything as literal text.

Why are they important? Web development heavily involves manipulating text—formatting user input, querying databases, building HTML outputs, and handling JSON. The functions below are essential for cleaning, searching, and modifying text data.

Function & Syntax	How it Works & Why Use It
`strlen(string $string)`	Returns the length of a string. Used to validate input (e.g., ensuring a password is at least 8 characters long).
`strpos(string $haystack, string $needle)`	Finds the position of the first occurrence of a substring. Returns `false` if not found. Crucial for checking if a string contains a specific word or character.
`str_replace($search, $replace, $subject)`	Replaces all occurrences of the search string with the replacement string. Used to sanitize inputs or swap out template tags for real data.
`strtolower($string)` `strtoupper($string)`	Converts a string to entirely lowercase or uppercase. Essential for standardizing data before saving it to a database or comparing strings without case sensitivity.
`substr($string, $start, $length)`	Extracts a portion of a string starting at a specified index. Used to grab excerpts (e.g., the first 50 characters of a blog post).
`trim(string $string)`	Removes whitespace (or other characters) from the beginning and end of a string. Always run this on user input from form submissions to prevent accidental blank spaces.

String Functions Example

```
<?php
```

3. Arrays in PHP (Part 1: Basics & Modification)

What are Arrays? An array is a special variable that can hold more than one value at a time. Unlike a string that holds one piece of text, an array acts like a list or a dictionary. PHP arrays are incredibly flexible and come in three main types:

- **Indexed Arrays:** Arrays with a numeric index, starting at 0.
- **Associative Arrays:** Arrays with named keys (like a dictionary).
- **Multidimensional Arrays:** Arrays containing one or more arrays.

Why are they important? Arrays hold the bulk of data in a PHP application. Database queries return arrays of rows, APIs consume and return arrays (via JSON), and HTML forms submit data as arrays.

```
<?php
// Indexed Array
$colors = ["Red", "Green", "Blue"];
echo $colors[0]; // Outputs: Red

// Associative Array
$user = [
    "name" => "John Doe",
    "role" => "Admin",
    "age"  => 30
];
echo $user["role"]; // Outputs: Admin
?>
```

Essential Array Functions (Modification & Search)

These functions are used daily to manipulate the contents of an array or check if specific data exists within it.

Function & Syntax	How it Works & Why Use It
`count(array $array)`	Returns the number of elements in an array. Used to check if an array is empty before looping through it, or to display total item counts.
`in_array($needle, $haystack)`	Checks if a value exists in an array. Returns true/false. Crucial for validation (e.g., checking if a user's role is in an array of allowed roles).
`array_key_exists($key, $array)`	Checks if a specific key exists in an associative array. Used to ensure a key is present before trying to access it, preventing undefined index errors.
`array_push($array, $val1, $val2...)`	Adds one or more elements to the end of an indexed array. Used to build lists dynamically, like adding items to a shopping cart array.
`array_merge($array1, $array2)`	Merges one or more arrays into one. If keys overlap in associative arrays, the later value overwrites the earlier one. Used to combine default settings with user settings.
`explode($separator, $string)`	Splits a string into an array by a specific character. Highly useful for breaking a comma-separated list into an array format.
`implode($glue, $pieces)`	The exact opposite of explode. Joins array elements into a single string with a specified separator.

Array Modification Example

```
<?php
$csvData = "apple,banana,orange";
```

4. Arrays in PHP (Part 2: Transformation & Iteration)

Advanced array functions allow developers to transform data cleanly without writing long, complex loops manually.

Function & Syntax	How it Works & Why Use It
`array_keys(array $array)`	Returns all the keys of an array as a new indexed array. Useful when you only need to know the properties of an object, not its values.
`array_values(array $array)`	Returns all the values of an array, re-indexing them numerically. Used to reset the numbering of an array after unsetting/removing elements.
`array_map($callback, $array)`	Applies a callback function to every element in an array and returns the modified array. Great for applying standard formatting (like capitalizing) to a whole list.
`array_filter($array, $callback)`	Iterates over the array passing them to a callback. If the callback returns true, the current value is kept. Used to strip empty values or filter specific data out.

5. Control Structures: Loops

What are Loops? Loops are used to execute the same block of code a specified number of times, or while a certain condition is true.

Why use them? They automate repetitive tasks. Instead of writing HTML for 50 table rows manually, you write one loop that runs 50 times pulling data from an array or database.

The For & While Loops

Used when you know exactly how many times a script should run (`for`), or when a loop should run as long as a condition is met (`while`).

```
<?php
// FOR LOOP syntax: for (init counter; test counter; increment counter)
for ($i = 1; $i <= 5; $i++) {
    echo "The number is: $i \n";
}

// WHILE LOOP syntax: while (condition is true)
$count = 1;
while ($count < 5) {
    echo "Still counting: $count \n";
    $count++; // Critical: prevents an infinite loop
}
?>
```

The Foreach Loop (The Developer's Favorite)

The `foreach` loop is specifically designed to work with arrays. It is the most common loop used in PHP because you rarely loop by simple numbers; you usually loop through sets of data.

```
<?php
$products = [
    "Laptop" => 999.99,
    "Mouse"  => 25.50,
    "Keyboard" => 45.00
];

// FOREACH syntax: foreach ($array as $key => $value)
```

6. User-Defined Functions

While PHP has hundreds of built-in functions (like `strlen`), you will write your own functions to bundle repetitive code into reusable blocks. A good function does one specific job and returns a result.

```
<?php
// Syntax: function functionName(type $parameter): returnType { ... }
function calculateTotal(float $price, float $taxRate = 0.05): float {
    $taxAmount = $price * $taxRate;
    $total = $price + $taxAmount;
    return $total;
}

// Calling the function
$cartTotal = calculateTotal(100.00); // Uses default 0.05 tax
echo "Total to pay: $" . $cartTotal; // Outputs 105.00
?>
```

7. Superglobals (Crucial for Web Dev)

Superglobals are built-in arrays that are always accessible, regardless of scope. They are how PHP interacts with the web browser and the server environment.

Superglobal Variable	Purpose and Usage
`$_POST`	An associative array of variables passed via the HTTP POST method. Used to securely collect form data (like passwords or long text).
`$_GET`	An associative array of variables passed via URL parameters. Used for pagination or filtering (e.g., `page.php?id=5` populates `$_GET['id']`).
`$_SESSION`	Variables stored on the server for a specific user across multiple pages. Used to keep users logged in or maintain shopping cart state.
`$_SERVER`	Contains information about headers, paths, and script locations. Used to find out the user's IP address, request method, or document root.

Developer Best Practices Checklist

- **Never trust user input.** Always sanitize data coming from `$_POST` or `$_GET` before using it or saving it to a database to prevent SQL Injection and XSS attacks.
- **Use strict comparison.** Whenever possible, use `===` instead of `==`. Strict comparison checks both value AND data type, preventing bugs caused by PHP's type juggling.
- **Handle Errors.** Use `try { } catch() { }` blocks when performing risky operations (like database connections) so your script doesn't crash fatally on the user's screen.